

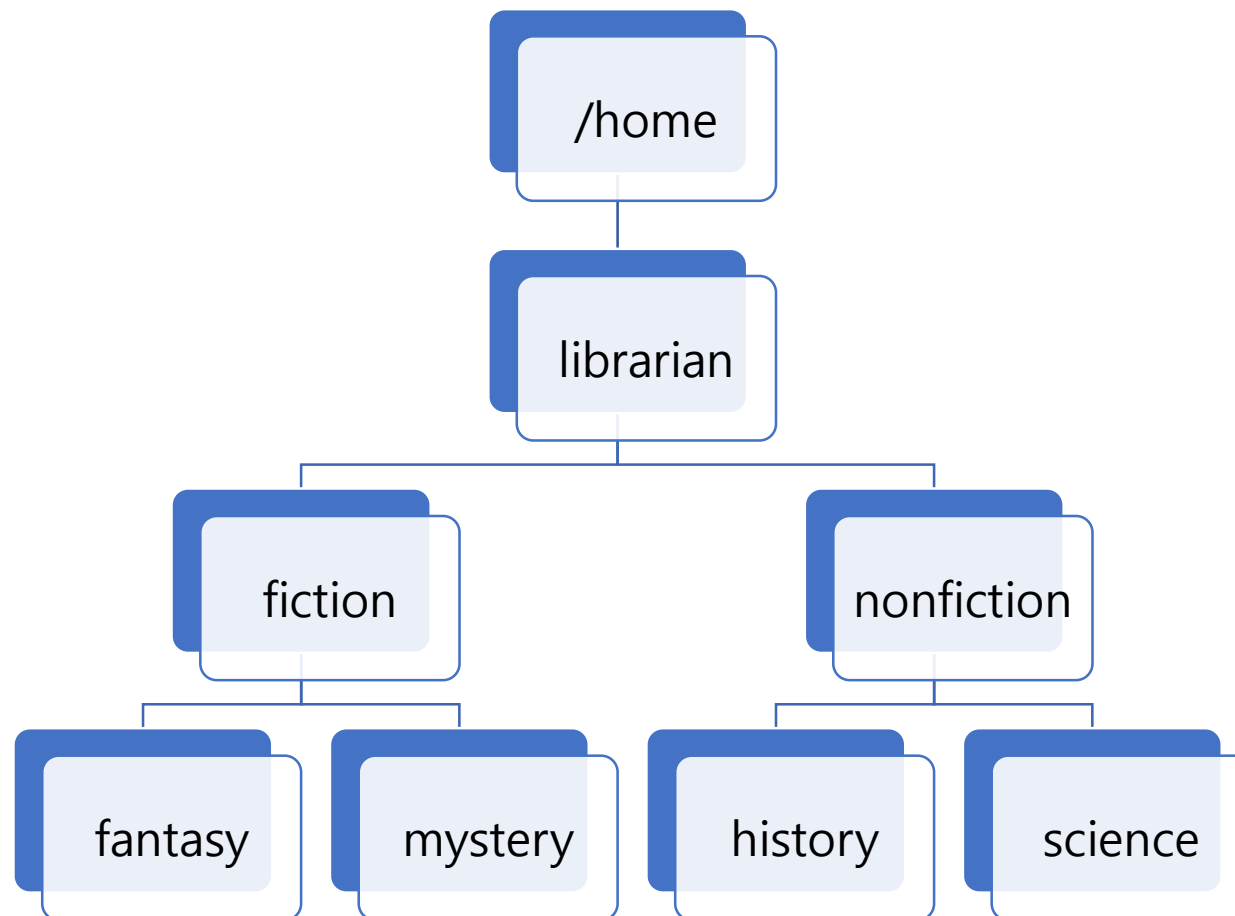
리눅스(Linux)

기본 명령어



≡ 디렉토리 구조 개요

리눅스 실습시 구현할 디렉토리 구조는 다음과 같습니다.



≡ 트리 형식으로 디렉토리 보기

디렉토리 구조를 tree 형태로 확인하기 위하여 tree 유틸리티를 먼저 설치하도록 합니다.

```
sudo apt install -y tree
```

```
# 전체 디렉토리 구조를 tree 형식으로 출력해 봅니다.
```

```
tree / -L 1
```

```
/
```

```
|—— bin -> usr/bin
```

```
|—— bin.usr-is-merged
```

```
|—— boot
```

```
|—— cdrom
```

```
... 중략
```

```
|—— swap.img
```

```
|—— sys
```

```
|—— tmp
```

```
|—— usr
```

```
|—— var
```

≡ 간단 명령어 보기

사용자 이름 및 도움말 명령어 등 간략하게 명령어를 살펴 봅니다.

Unix name를 확인합니다.

uname

Linux

"manual"의 약자로, 명령어나, 라이브러리 함수, 파일 형식 등에 대한 문서를 확인할 수 있도록 도와줍니다.

단축 키 : 밑으로 이동(j 누름), 위로 이름(k 누름), 빠져 나가기(q)

man uname

도움말을 간단히 보려면 다음과 같이 입력합니다.(주의 : 빼기 2개로 표현)

uname --help

pwd 명령어는 현재 디렉토리를 확인할 수 있습니다.

pwd

디렉토리를 home으로 이동합니다.

cd /home

≡ 디렉토리 생성 및 이동하기

샘플 디렉토리들을 생성하고, 이동하는 방법에 대하여 살펴 보도록 합니다.

홈 디렉토리로 이동합니다.

`cd`

nonfiction 디렉토리를 신규로 생성합니다.

`mkdir /home/librarian/nonfiction`

nonfiction 디렉토리로 이동합니다.

`cd /home/librarian/nonfiction`

pwd 명령어로 현재 디렉토리를 확인합니다.

`pwd`

nonfiction 디렉토리 하위에 무한 도전(science) 디렉토리를 생성하고, 이동합니다.

`mkdir /home/librarian/nonfiction/science`

`cd /home/librarian/nonfiction/science`

`pwd`

`# /home/librarian/nonfiction/science`

≡ 디렉토리 생성 및 이동하기

샘플 디렉토리들을 생성하고, 이동하는 방법에 대하여 살펴 보도록 합니다.

```
# nonfiction 디렉토리 하위에 `역사`(history) 디렉토리를 생성해 봅니다.
```

```
mkdir /home/librarian/nonfiction/history
```

```
cd /home/librarian/nonfiction/history
```

```
# 사용자의 홈(home) 디렉토리로 다시 이동합니다.
```

```
cd # 엔터키 누름
```

```
# 하위에 fiction 디렉토리를 신규로 생성합니다.
```

```
mkdir /home/librarian/fiction
```

```
# fiction 디렉토리로 이동합니다.
```

```
cd /home/librarian/fiction
```

```
# fiction 디렉토리 하위에 환타지(fantasy) 디렉토리를 생성하고, 이동합니다.
```

```
mkdir /home/librarian/fiction/fantasy
```

```
cd /home/librarian/fiction/fantasy
```

```
pwd
```

≡ 디렉토리 생성 및 이동하기

샘플 디렉토리들을 생성하고, 이동하는 방법에 대하여 살펴 보도록 합니다.

fiction 디렉토리 하위에 미스터리(mystery) 디렉토리를 생성해 봅니다.

```
mkdir /home/librarian/fiction/mystery
```

```
cd /home/librarian/fiction/mystery
```

디렉토리 구조 개요

풀어 보시다.

다음과 같은 디렉토리 구조를 구현해 보세요.

구조 설명

/home/bookadmin:

책을 관리하는 사용자의 홈 디렉토리

fiction: 소설(픽션) 분야

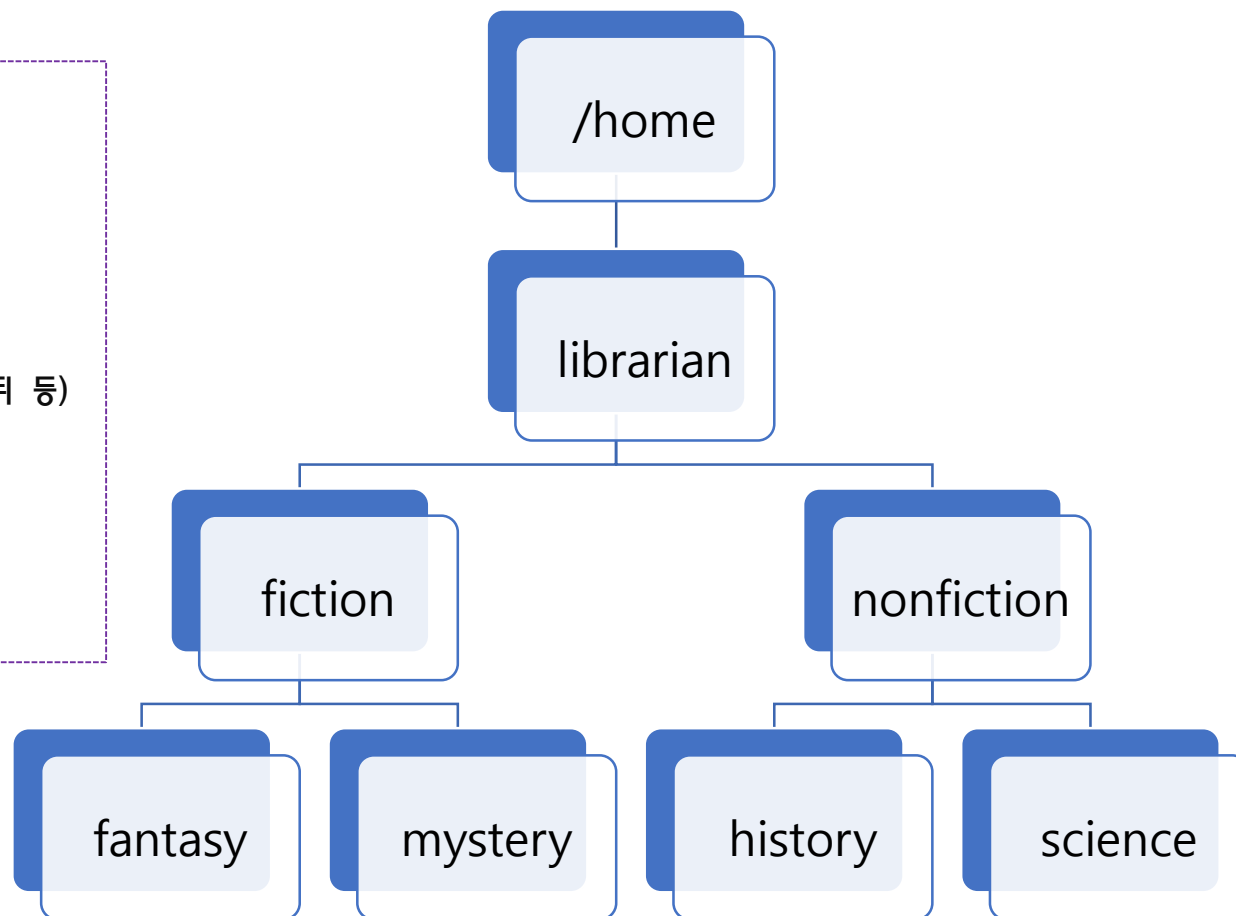
fantasy: 판타지 소설 (예: 해리 포터, 반지의 제왕 등)

mystery: 추리/미스터리 소설 (예: 셜록 홈즈, 애거사 크리스티 등)

nonfiction: 논픽션 분야

history: 역사 관련 서적

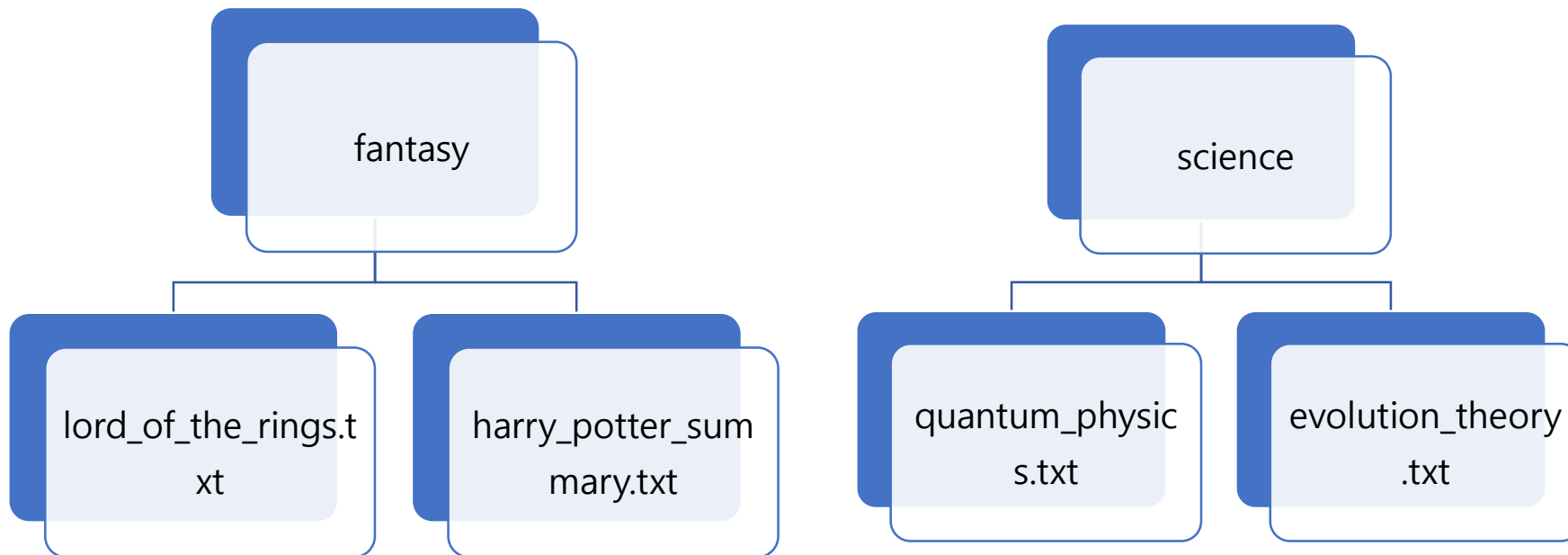
science: 과학 관련 서적



파일 생성하기

기존 디렉토리 구조에 다음과 같이 파일 및 디렉토리를 구성해 봅니다.

실습 구성도



≡ 파일 생성하기

파일을 생성하는 명령어와 파일의 내용을 확인하는 명령어를 살펴 봅니다.

사용자 home 디렉토리로 이동합니다.

`cd`

echo 명령어는 터미널에서 문자열을 출력하는 데 사용되는 기본적인 명령어입니다.

`echo "Hello, World!"`

Hello, World!

지금까지의 실습은 `절대 경로를 이용한 방식`이었고, 아래의 실습은 `상대 경로를 이용한 방식`입니다.

다음은 fantasy 디렉토리 내에 lord_of_the_rings.txt 라는 파일을 신규 생성합니다.

`echo "This is a test" > fiction/fantasy/lord_of_the_rings.txt`

lord_of_the_rings.txt 파일의 내용을 확인합니다.

`cat fiction/fantasy/lord_of_the_rings.txt`

This is a test

동일한 방식으로 다음 파일을 생성합니다.

`echo "I will be back" > fiction/fantasy/harry_potter_summary.txt`

파일 생성하기

파일을 생성하는 명령어와 파일의 내용을 확인하는 명령어를 살펴 봅니다.

```
# 현재 디렉토리는 사용자의 home 디렉토리입니다.
```

```
pwd
```

```
/home/librarian
```

```
# touch 명령어는 파일이 존재하지 않을 경우 크기가 0인 새로운 파일을 생성해 줍니다.
```

```
touch nonfiction/science/quantum_physics.txt
```

```
touch nonfiction/science/evolution_theory.txt
```

```
# 해당 폴더의 파일 내용을 확인하면서, 파일의 크기도 동시에 확인하도록 합니다.
```

```
ls -al nonfiction/science/
```

```
# total 8
```

```
# drwxrwxr-x 2 librarian librarian 4096 Apr 22 03:34 .
```

```
# drwxrwxr-x 4 librarian librarian 4096 Apr 22 03:29 ..
```

```
# -rw-rw-r-- 1 librarian librarian    0 Apr 22 03:33 quantum_physics.txt
```

```
# -rw-rw-r-- 1 librarian librarian    0 Apr 22 03:33 evolution_theory.txt
```

≡ 디렉토리 구조 확인

tree 명령어를 사용하면 디렉토리의 구조를 파악할 수 있습니다.

tree 명령어를 사용하여 디렉토리의 구조를 파악합니다.

tree /home/librarian

/home/librarian

├── fiction

│ ├── fantasy

│ │ ├── lord_of_the_rings.txt

│ │ └── harry_potter_summary.txt

│ └── mystery

└── nonfiction

├── history

└── science

├── quantum_physics.txt

└── evolution_theory.txt

≡ 파일 목록 보기

파일 목록을 확인하는 다양한 방법에 대하여 살펴 봅니다.

ls 명령어를 사용하여 디렉토리의 구조를 파악합니다.

`pwd`

`/home/librarian`

`ls` # 파일이나 디렉토리의 목록을 보여 줍니다.

`fiction` `nonfiction`

`ls -l` # 자세한 정보와 함께 보여 줍니다.

`total 8`

`drwxrwxr-x 4 librarian librarian 4096 Dec 13 01:42 fiction`

`drwxrwxr-x 4 librarian librarian 4096 Dec 13 01:44 nonfiction`

≡ 파일 목록 보기

파일 목록을 확인하는 다양한 방법에 대하여 살펴 봅니다.

자세한 정보와 함께 보여 주되, 숨겨져 있는 항목도 같이 보여 줍니다.

결과 물을 보면 앞에 . 기호가 있으면 hidden 목록입니다.

`ls -al`

total 48

drwxr-x--- 6 librarian librarian 4096 Dec 13 01:18 .

drwxr-xr-x 3 root root 4096 Dec 12 07:16 ..

-rw----- 1 librarian librarian 240 Dec 12 09:05 .bash_history

drwxrwxr-x 4 librarian librarian 4096 Dec 13 01:42 fiction

drwxrwxr-x 4 librarian librarian 4096 Dec 13 01:44 nonfiction

... 이하 중략

파일 목록 보기

파일 목록을 확인하는 다양한 방법에 대하여 살펴 봅니다.

```
ls -l /home/librarian/nonfiction/
```

R 옵션을 사용하면 하위 경로와 그 안에 있는 모든 파일들도 같이 나열해 줍니다.

```
ls -lR /home/librarian/nonfiction/
```

```
/home/librarian/nonfiction/:
```

```
total 8
```

```
drwxrwxr-x 2 librarian librarian 4096 Dec 13 01:44 history
```

```
drwxrwxr-x 2 librarian librarian 4096 Dec 13 02:39 science
```

```
/home/librarian/nonfiction/history:
```

```
total 0
```

```
/home/librarian/nonfiction/science:
```

```
total 0
```

```
-rw-rw-r-- 1 librarian librarian 0 Dec 13 02:39 quantum_physics.txt
```

```
-rw-rw-r-- 1 librarian librarian 0 Dec 13 02:39 evolution_theory.txt
```

≡ 파일 목록 보기

파일 목록을 확인하는 다양한 방법에 대하여 살펴 봅니다.

F 옵션을 사용하면 파일 형식을 알리는 문자를 각 파일 뒤에 추가합니다.

`ls -lF /etc`

total 912

drwxr-xr-x 2 root root 4096 Aug 27 14:26 alternatives/

drwxr-xr-x 2 root root 4096 Aug 27 14:21 apparmor/

drwxr-xr-x 9 root root 4096 Aug 27 14:26 apparmor.d/

-rwxr-xr-x 1 root root 243 Aug 27 14:26 nftables.conf*

lrwxrwxrwx 1 root root 27 Aug 27 14:21 localtime -> /usr/share/zoneinfo/Etc/UTC

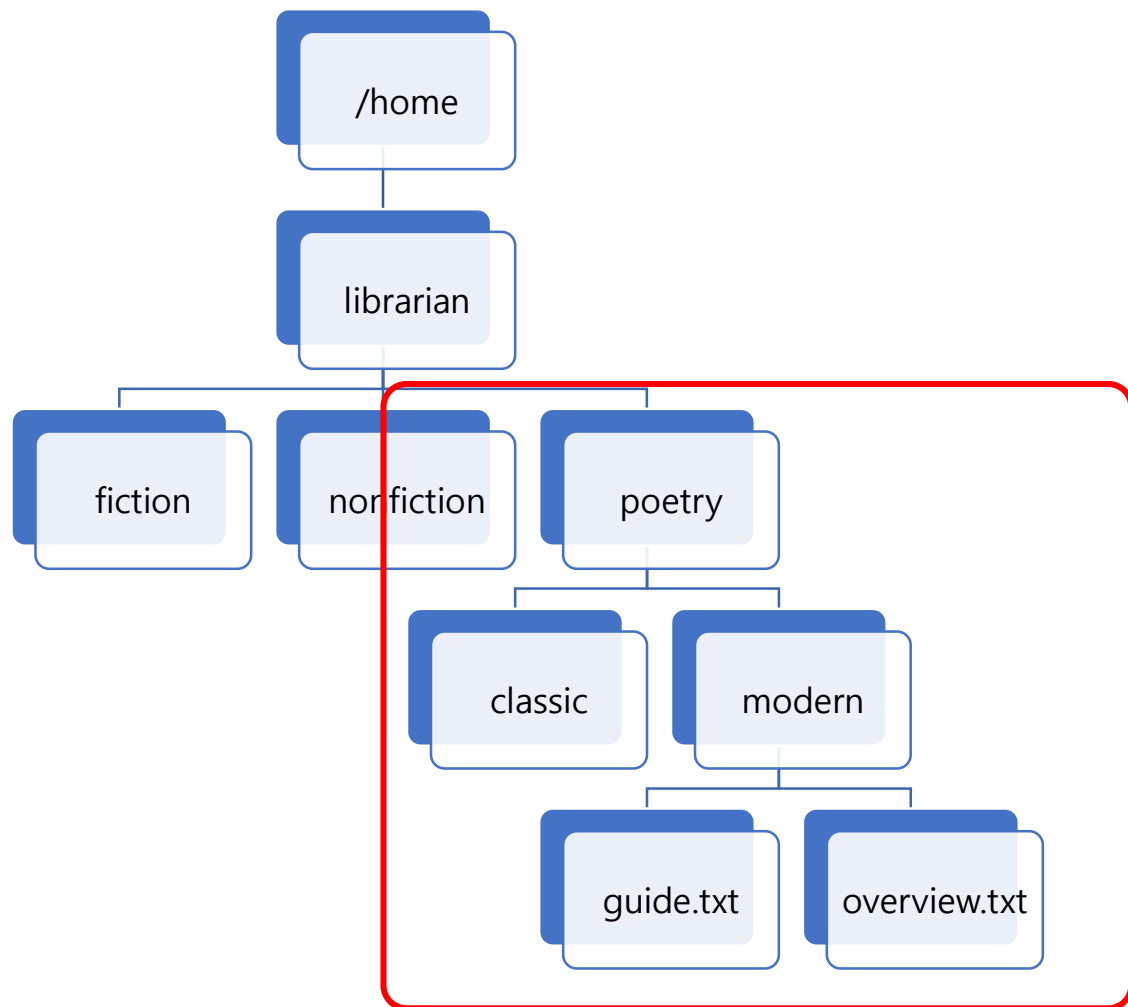
-rw-r--r-- 1 root root 1136 Aug 27 14:26 crontab

-rw-r--r-- 1 root root 54 Aug 27 14:21 crypttab

... 이하 중략

디렉토리 구성도

기존 디렉토리 구조에 다음과 같이 poetry 디렉토리 구조를 생성해 보도록 합니다.



디렉토리 및 파일 생성

다양한 방식으로 디렉토리 및 파일을 생성하는 방법에 대하여 살펴 보겠습니다.

디렉토리를 home/librarian으로 다시 이동합니다.

```
cd /home/librarian
```

poetry 디렉토리 하위 내에 classic 디렉토리를 신규로 생성합니다.(오류 발생)

```
mkdir /home/librarian/poetry/classic
```

mkdir: cannot create directory '/home/librarian/poetry/classic' : No such file or directory

디렉토리 생성시 존재하지 않는 상위 디렉토리까지 생성하려면 -p 옵션을 사용하면 됩니다.

```
mkdir -p /home/librarian/poetry/classic
```

동일한 방식으로 modern 디렉토리를 생성하도록 합니다.

```
mkdir -p /home/librarian/poetry/modern
```

guide.txt 파일과 overview.txt 파일을 생성하도록 합니다.

```
echo "poetry man in black box file" > poetry/modern/guide.txt
```

```
echo "some settings be effected" > poetry/modern/overview.txt
```

≡ 트리 구조 확인

전체 디렉토리 구조를 tree 형식으로 출력해 봅니다.

전체 디렉토리 구조를 tree 형식으로 출력해 봅니다.

tree

```
.
├── fiction
│   ├── fantasy
│   │   ├── lord_of_the_rings.txt
│   │   └── harry_potter_summary.txt
│   └── mystery
├── nonfiction
│   ├── history
│   └── science
│       ├── quantum_physics.txt
│       └── evolution_theory.txt
└── poetry
    ├── modern
    │   ├── overview.txt
    │   └── guide.txt
    └── classic
```

복사 및 이동

특정 디렉토리의 파일을 복사하거나 이동하는 명령어에 대하여 살펴 보도록 하겠습니다.

다음 요구 사항대로 파일을 복사하거나 이동 실습을 진행해 보세요.

cd

복사01 : poetry/modern/* → fiction/mystery/

cp poetry/modern/* fiction/mystery/

tree # poetry/modern/ 폴더의 모든 내용이 fiction/mystery/ 폴더에 복사가 되었는 지 확인하도록 합니다.

...중략

```
|   └── mystery
|       ├── overview.txt
|       └── guide.txt
```

...중략

```
└── poetry
    ├── modern
    │   ├── overview.txt
    │   └── guide.txt
```

≡ 복사 및 이동

특정 디렉토리의 파일을 복사하거나 이동하는 명령어에 대하여 살펴 보도록 하겠습니다.

복사02 : nonfiction/science → fiction/classic/로 복사하되 파일 이름에 접두사 "fiction_"를 붙여 주세요.

```
cp nonfiction/science/quantum_physics.txt fiction/mystery/fiction_quantum_physics.txt
```

```
cp nonfiction/science/evolution_theory.txt fiction/mystery/fiction_evolution_theory.txt
```

이동01 : poetry/modern/overview.txt → nonfiction/history/으로 이동합니다.

```
mv poetry/modern/overview.txt nonfiction/history/
```

이름 변경 01 : poetry/modern/guide.txt → poetry/modern/abandon.txt

```
mv poetry/modern/guide.txt poetry/modern/abandon.txt
```

≡ 파일 찾기

여러 가지 조건을 사용하여 파일을 찾아 보도록 합니다.

최상위 디렉토리로 이동합니다.

```
cd /
```

특정 디렉토리에 이름이 overview.txt인 파일 찾기

```
find /home/librarian -name "overview.txt"
```

```
# /home/librarian/fiction/mystery/overview.txt
```

대소문자 구분 없이 찾기

```
find /home/librarian -iname "WORLD.TXT"
```

```
# /home/librarian/fiction/fantasy/harry_potter_summary.txt
```

파일 이름이 k로 시작하고, 확장자가 txt인 항목들 찾기

```
find /home/librarian -name "k*.txt"
```

```
# /home/librarian/fiction/mystery/fiction_evolution_theory.txt
```

```
#
```

```
/home/librarian/fiction/mystery/fiction_quantum_physics.txt
```

m으로 시작하는 디렉토리만 찾기

```
find /home/librarian -type d -name "m*"
```

```
# /home/librarian/poetry/modern
```

```
# /home/librarian/poetry/classic
```

```
# /home/librarian/nonfiction
```

```
# /home/librarian/fiction/mystery
```

특정 크기 이상/이하의 파일 찾기

```
find /home/librarian -size +20m # 20MB보다 큰 파일
```

```
find /home/librarian -size -100k # 100KB보다 작은 파일
```

≡ 파일 찾기

여러 가지 조건을 사용하여 파일을 찾아 보도록 합니다.

최근 수정된 파일 찾기

`find /home/librarian -mtime -1` # 1일 이내 수정된 파일

`find /home/librarian -mtime +7` # 7일 이상 된 파일

-mtime은 수정일 기준 (-1: 하루 이내, +7: 7일 초과)

실행 권한 있는 파일 찾기

`find /home/librarian -type f -executable`

리눅스(Linux)

VI 편집기



≡ 파일 생성

vi 편집기를 사용하여 텍스트 파일을 생성하는 방법에 대하여 살펴 봅니다.

vi 편집기를 이용하여 실습 후 저장합니다.

`vi react.txt`

class

props

state

useState

useEffect

jsx

render

event

hook

context

redux

esc 키를 누르고, :wq 명령어로 종료합니다.

`cat react.txt` # 파일 내용 확인하기

다음 파일을 만들어 주세요.

`vi oracle.txt`

select

insert

update

delete

where

joint

group by

having

commit

rollback

esc :wq 으로 종료합니다.

`cat oracle.txt` # 파일 내용 확인하기

파일 수정

vi 편집기를 사용하여 텍스트 파일을 수정하는 방법에 대하여 살펴 봅니다.

```
cp react.txt react.bak
```

```
cp oracle.txt oracle.bak
```

```
# 명령 모드의 커서 이동 실습
```

```
# react.txt 파일을 다시 엽니다.
```

```
vi react.txt
```

명령어	설명	명령어	설명
j	Up	0	Home
k	Down	\$	End
h	Left	gg	Ctrl + Home
l(엘)	Right	G	Ctrl + End

```
# 복사 및 삭제 테스트 및 undo 테스트
```

```
# react.txt 파일을 다시 엽니다.
```

```
vi react.txt
```

```
# [dd]를 사용하여 첫 줄을 삭제해 봅니다.
```

```
# [u]를 사용하여 이전 내용을 취소합니다.
```

```
# [3dd]를 사용하여 첫 3줄을 삭제해 봅니다.
```

```
# [u]를 사용하여 이전 내용을 취소합니다.
```

```
# [yy]를 사용하여 첫 줄을 복사합니다.
```

```
# 가장 아래로 이동하여 [p]를 사용하여 붙여 넣습니다.
```

```
# [u]를 사용하여 이전 내용을 취소합니다.
```

```
# 가장 위로 이동하여 [3yy]를 사용하여 복사합니다.
```

```
# 가장 아래로 이동하여 [p]를 사용하여 붙여 넣습니다.
```

```
# [u]를 사용하여 이전 내용을 취소합니다.
```

리다이렉션

입력(input)이나 출력(output)의 기본 흐름을 변경하는 것을 의미합니다.

```
cp react.bak react.txt
```

리다이렉션 실습

react.txt 파일의 내용을 summary.txt 파일에 추가합니다.

summary.txt 파일은 신규 생성이 되고, > 기호에 의하여

summary.txt 파일에 react.txt 파일의 내용 덮어 쓰기 됩니다.

```
cat react.txt > summary.txt
```

```
cat summary.txt # react.txt 파일의 내용이 보입니다.
```

oracle.txt 파일의 내용을 summary.txt 파일에 추가합니다.

```
cat oracle.txt > summary.txt
```

```
cat summary.txt # oracle.txt 파일의 내용이 보입니다.
```

```
cat react.txt > summary.txt # react.txt 파일로 다시 덮어 쓰기
```

```
cat oracle.txt >> summary.txt # oracle.txt 파일의 내용 누적
```

summary.txt 파일은 react.txt + oracle.txt 파일의 합쳐진 내용이 출력됩니다.

```
cat summary.txt
```

입력 리다이렉션

```
cat < summary.txt
```

리다이렉션

입력(input)이나 출력(output)의 기본 흐름을 변경하는 것을 의미합니다.

```
echo `redux` >> summary.txt  
echo `render` >> summary.txt
```

```
echo `hook` >> summary.txt  
echo `redux` >> summary.txt
```

```
# 마지막 5줄 출력하기  
tail -n 5 summary.txt
```

```
# 찾기 및 치환하기 테스트  
vi summary.txt
```

```
# :set nu 라인 번호를 보여 주도록 합니다. :set nonu
```

```
# redux 단어를 찾기 위하여 `/redux` 엔터 키를 누릅니다.  
# 여러 단어가 검색된 경우, 다음 redux를 검색하려면 `esc` 키를 누른  
후 `n` 키를 계속 누르면 됩니다.(↔N)
```

```
# 11번 라인의 redux라는 글자를 switch로 치환(substitute)합니다.
```

```
:11s/redux/switch/
```

```
# 모든 라인의 redux라는 글자를 hello로 치환합니다.
```

```
# %는 전체 범위를 의미
```

```
# global 플래그는 각 줄에서 모든 일치하는 항목을 전부 치환하라는 의미
```

```
:%s/redux/hello/g
```

```
# 14번에서 24번줄까지 hook이라는 글자를 case로 치환하되, 변경 작업을 진  
행할지 사용자에게 물어 봅니다.
```

```
:14,24s/hook/case/gc
```

≡ 리다이렉션

입력(input)이나 출력(output)의 기본 흐름을 변경하는 것을 의미합니다.

11번째 줄 아래 line에 다음 실행 결과를 추가하겠습니다.

```
:11r! ls -al /home/librarian/
```

esc 키, :wq 명령어로 종료 후, 잘 반영이 되었는 지 확인합니다.

```
cat summary.txt
```

파일을 열어서 render 1개를 breaking으로 변경하도록 합니다.

그리고, 끝 줄에 대문자 "BREAK"을 추가합니다.

```
vi summary.txt
```

grep은 특정 패턴을 찾을 때 사용하는 명령어입니다.

특정 라인에 render라는 글자가 들어 있는 행을 알려 줍니다.

```
grep "render" summary.txt
```

대소문자 구분 없이 찾고 싶을 때

```
grep -i "render" summary.txt
```

해당 단어가 포함된 줄 번호도 함께 출력하고 싶을 때:

```
grep -in "render" summary.txt
```

정확히 단어 단위로 "render"만 찾고 싶을 때 (예: "breaking" 제외):

```
grep -wn "render" summary.txt
```

```
more summary.txt
```

```
more -n 10 summary.txt
```

≡ more 명령어

내용이 많은 파일을 화면 단위로 끊어서 출력하고자 하는 경우 사용합니다.

```
# 3번 반복하면서, 라인에 숫자형 번호를 붙여서 신규 파일 작성
for i in {1..3}; do cat summary.txt; done | nl >> multitextfile.txt

head -n 5 multitextfile.txt

tail -n 3 multitextfile.txt
```

```
# 화면 단위로 출력하기
cat multitextfile.txt | more

[Enter] 키, [SpaceBar] 키, `q` 키를 각각 눌러 보도록 합니다.
```

≡ `diff 명령어

파일 2개의 내용을 비교하는 명령어입니다.

```
# 3번 반복하면서, 라인에 숫자형 번호를 붙여서 신규 파일 작성
cp react.txt react.ppt
```

```
vi react.ppt
```

```
# 모든 라인의 state라는 글자를 anything라고 치환합니다.
```

```
:%s/state/anything/g
```

```
:%s/useState/sundori/g
```

```
# :wq 저장 후 종료
```

```
# -brief 옵션은 단지 값이 동일한지 체크합니다.
```

```
diff -brief react.txt react.ppt
```

```
# Files react.txt and react.doc differ
```

```
# -brief 옵션은 단지 값이 동일한지 체크합니다.
```

```
diff react.txt react.ppt
```

리눅스(Linux)

User Account Management



☰ User Account Management

☰ 사용자와 그룹 생성하기

사용자(User)와 그룹(Group)을 생성하는 실습입니다.

그룹 목록

editorteam



broadcast



pressdesk



mediacrew



사용자 목록



starshine



cloudnine



thunderfox



fireball

☰ User Account Management

☰ starshine 사용자 계정 생성하기

starshine라는 사용자를 생성하면서 이와 관련된 파일들을 하나씩 살펴 보기로 합니다.

```
# starshine이라는 사용자를 생성하면서 이와 관련된 파일들을 하나씩 살펴 보기로 합니다.  
# starshine이라는 사용자 계정 생성  
sudo su -  
[sudo] password redux librarian: 비번 입력  
  
cd /etc/skel/ # 신규 사용자 생성시 사용자의 홈 디렉토리에 복사될 템플릿 디렉토리  
# 신규 사용자에게 다음 index.html 파일을 기본으로 제공하기 위해 작성합니다.  
echo 'welcome~~my homepage' > index.html  
  
cat index.html  
  
cd /
```

```
# 신규 사용자를 추가시 홈 디렉토리를 생성(-m 옵션)해 줍니다.  
useradd -m starshine  
passwd starshine # starshine 사용자 비번 변경 명령어  
New password: skywalker456 ← 눈에 보이지 않음  
Retype new password: skywalker456 ← 눈에 보이지 않음  
passwd: password updated successfully  
  
# MobaXTerm에서 starshine로 접속해 보기
```

항목	설명
사용자	starshine
비밀 번호	skywalker456

≡ User Account Management

≡ starshine 사용자 계정 생성하기

starshine라는 사용자를 생성하면서 이와 관련된 파일들을 하나씩 살펴 보기로 합니다.

```
# starshine 이라는 사용자를 생성하였습니다.  
# 이후 리눅스 서버에서 어떠한 작업이 이루어 졌는지를 살펴보도록 합니다.  
tail /etc/passwd | grep starshine  
# starshine:x:1001:1001::/home/starshine:/bin/sh  
# ← /etc/passwd 파일 내에 id, 암호, UID, GID, 이름, 홈 디렉토리,  
# 사용 셸이 차례로 등록이 된 것을 알 수 있습니다.  
# 참고 사항  
# uid : 유저의 id(시스템이 인식하는 번호)  
# gid : 그룹의 id(디폴트로 uid 이름과 동일한 이름이 생성이 됩니다.)  
  
# /etc/passwd 파일 : 사용자를 생성하면 해당 정보를 담고 있는 가장 핵심적인 파일입니다.  
# /etc/shadow 파일 내에 starshine의 패스워드와 aging 정보 등록  
tail /etc/shadow | grep starshine  
# starshine:$y$j9T$h32XwFXT20q4JXbz2lxzF0$Yq/2cOS18BUMH//  
# zk8c.yuJmVFRKYc6WN3PffQc7Mg6:20073:0:99999:7:::
```

☰ User Account Management

☰ starshine 사용자 계정 생성하기

starshine라는 사용자를 생성하면서 이와 관련된 파일들을 하나씩 살펴 보기로 합니다.

```
# /etc/group 파일 내에 starshine의 패스워드와 aging 정보 등록
tail /etc/group | grep starshine
# starshine:x:1001:
# 즉, starshine의 계정 이름과 동일한 starshine이라는 그룹 명과
# GID 1001이라는 그룹이 생성되어 /etc/group 파일 내에 등록이 되었습니다.

# starshine의 홈 디렉토리 생성 및 환경 파일 생성
사용자가 만들어지고 나면, 사용자의 홈 디렉토리가 생성이 됩니다.
특별히 값을 변경하지 않았다면 starshine의 홈 디렉토리는 /home/starshine입니다.
ls -al /home/starshine

# 이 디렉토리에 index.html 문서의 내용을 확인합니다.
cat /home/starshine/index.html
```

☰ User Account Management

☰ starshine 사용자 계정 생성하기

starshine라는 사용자를 생성하면서 이와 관련된 파일들을 하나씩 살펴 보기로 합니다.

참고로 위의 파일들은 /etc/skel 이라는 디렉토리에 존재하는 모든 파일 들이 복사되어 만들어진 내용입니다.

skel은 사용자 생성시 홈 디렉토리에 넣어줄 파일 목록을 저장하고 있는 일종의 템플릿(구조, 뼈대) 디렉토리입니다.

참고로 모든 파일은 숨김 파일이므로 ls 명령어 사용시 -a 옵션을 사용하도록 합니다.

`ls -al /etc/skel`

useradd 명령어가 참조하는 파일 : /etc/login.defs

/etc/login.defs 파일은 useradd 명령어가 새로운 계정을 생성할 때 반드시 참조하는 파일입니다.

≡ User Account Management

≡ starshine 사용자 계정 생성하기

starshine라는 사용자를 생성하면서 이와 관련된 파일들을 하나씩 살펴 보기로 합니다.

useradd 명령어가 참조하는 파일은 /etc/default/useradd입니다.

이 파일을 "useradd의 기본 파일" 이라고 하며, useradd 계정 생성시에 어떤 환경과 어떤 파일들을 참조하여 새로운 계정을 생성할 것인가에 대하여 정의를 하고 있는 파일입니다.

즉, 이 파일의 값을 바꾸게 되면 기본 설정 값이 바뀌게 된다는 얘기입니다.

`cat /etc/default/useradd`

하단의 /etc/default/useradd 파일 내용을 참조 바랍니다.

/etc/default/useradd 파일 내용

GROUP=100 //기본 등록 그룹의 GID

HOME=/home //생성될 홈 디렉토리의 위치

INACTIVE=-1 //패스워드 종료일 이후의 유효(기간) 여부 설정(0, -1, 1)

EXPIRE= //계정 종료 일자 지정

SHELL=/bin/bash //기본 사용 셸 지정

SKEL=/etc/skel //홈 디렉토리에 복사할 기본 환경 파일의 위치

CREATE_MAIL_SPOOL=yes // 메일 저장소 파일(mail spool) 생성 여부

☰ User Account Management

☰ cloudnine 사용자 계정 생성하기

다음 요구 사항을 충족하는 사용자를 생성하시오.

구현할 내용

항목	설명
사용자	cloudnine
홈 디렉토리	/home/cloudnine
UID	7000
shell	/bin/bash

```
# 옵션을 지정하면, 기본적인 설정 값에 우선하여 사용자가 생성됩니다.  
useradd -m -d /home/cloudnine -u 7000 -s /bin/bash cloudnine
```

```
# 내용 점검하기
```

```
tail /etc/passwd | grep cloudnine
```

```
# cloudnine:x:7000:7000::/home/cloudnine:/bin/bash
```

```
tail /etc/group | grep cloudnine
```

```
# cloudnine:x:7000:
```

```
ls -al /home/cloudnine
```

☰ User Account Management

☰ thunderfox 사용자 계정 생성하기

다음 요구 사항을 충족하는 사용자를 생성하시오.

구현할 내용

```
useradd -m -c tough_guy -e 2025-12-25 -d /home/thunderfox -u 7500 -s /bin/bash -p $(openssl passwd -6 test1234) thunderfox
```

```
tail /etc/passwd | grep thunderfox
```

```
# thunderfox:x:7500:7500:tough_guy:/home/thunderfox:/bin/bash
```

```
tail /etc/group | grep thunderfox
```

```
# thunderfox:x:7000:
```

```
ls -al /home/thunderfox
```

항목	설명
사용자	thunderfox
홈 디렉토리	/home/thunderfox
코멘트	tough_guy
계정 사용 종료일	2025-12-25
UID	7500
Shell	/bin/bash
비밀 번호	test1234

≡ User Account Management

≡ useradd -D 옵션

useradd -D 옵션은 useradd의 기본 환경 설정을 변경하는 역할을 합니다.

다음 명령어로 useradd의 실행 환경을 확인합니다.

useradd -D

GROUP=100

HOME=/home # 기억해 두세요.

INACTIVE=-1

EXPIRE=

SHELL=/bin/sh

SKEL=/etc/skel

CREATE_MAIL_SPOOL=no

LOG_INIT=yes

"useradd -D"는 /etc/default/useradd 파일의 값을 변경합니다.

사용자의 기본 홈 디렉토리를 변경하여 보자

useradd -D -b /myhome

cat /etc/default/useradd

GROUP=100

HOME=/myhome # 홈 디렉토리가 변경되었습니다.

INACTIVE=-1

EXPIRE=

SHELL=/bin/sh

SKEL=/etc/skel

CREATE_MAIL_SPOOL=no

LOG_INIT=yes

이렇게 변경이 된 후에는 useradd로 생성되는 사용자들의 홈 디렉토리는 /myhome/사용자ID가 됩니다.

useradd로 새로 생성될 사용자의 셸 변경하기

useradd -D -s /bin/sh

cat /etc/default/useradd

"기본 셸"의 정보(SHELL 항목)가 변경이 되었는 지를 확인합니다.

≡ User Account Management

≡ 사용자 계정 생성/삭제

useradd fireball라고 입력을 했을 때 다음과 같은 규칙으로 사용자를 생성해 보세요.

구현할 내용

다음과 같이 /etc/default/useradd 파일을 수정해 주세요.
그런 다음 '파이어볼' 사용자를 다음과 같이 생성해 보세요.

항목	설명
사용자	fireball
홈 디렉토리	/test
사용자 skel	/skelsam/
shell	tcsh

만일을 대비하여 useradd 파일을 백업합니다.

```
cp /etc/default/useradd /etc/default/useradd.bak
```

```
vi /etc/default/useradd
```

... 중략

```
SHELL=/bin/tcsh
```

... 중략

```
HOME=/test
```

```
SKEL=/skelsam
```

```
:wq
```

정규 표현식으로 여러 개 동시에 확인하기

```
cat /etc/default/useradd | grep 'tcsh\skelsam\test'
```

```
SHELL=/bin/tcsh
```

```
HOME=/test
```

```
SKEL=/skelsam/
```

≡ User Account Management

≡ 사용자 계정 생성/삭제

useradd fireball라고 입력을 했을 때 다음과 같은 규칙으로 사용자를 생성해 보세요.

관련 디렉토리를 생성하고, 파일을 복사합니다.

```
mkdir /test/
```

```
mkdir /skelsam/
```

skelsam 디렉토리는 skel 디렉토리의 내용과 동일합니다.

```
cp -r /etc/skel/. /skelsam/
```

index.html 파일 내용 수정하기

```
vi /skelsam/index.html
```

```
hello~~linux...world
```

```
:wq
```

fireball 사용자를 추가합니다.

```
useradd fireball
```

```
tail -1 /etc/passwd
```

fireball:x:7001:7001::/test/fireball:/bin/tcsh

위의 결과를 보면 파이어볼의 home 디렉토리는 /test 임이 확인됩니다.

그리고, 기본 shell이 tcsh임을 확인할 수 있습니다.

userdel -r fireball # 사용자 삭제하기

다음 내용은 오류는 아니고, 단순한 알람입니다.

존재하지 않아서 처리할 내용이 없다는 정도의 의미입니다.

```
userdel: fireball mail spool (/var/mail/fireball) not found
```

```
userdel: fireball home directory (/test/fireball) not found
```

≡ User Account Management

≡ 그룹 생성과 삭제

특정 이름의 새로운 그룹을 생성하고 삭제하는 테스트를 진행해 봅니다.

```
# 문제 : 특정 이름의 새로운 그룹 생성하기
# 다음 예시는 editorteam이라는 그룹을 생성합니다.
groupadd editorteam

# 문제 : 정의되어 있는 그룹 리스트를 확인합니다.
tail /etc/group | grep editorteam

# editorteam:x:7001: => 7001(상황에 따라 다름)는 GID입니다.
```

```
# 그룹 번호 7777로 broadcast이라는 그룹을 생성합니다.
# -g 옵션으로 그룹 번호를 지정할 수 있습니다.
groupadd -g 7777 broadcast
tail /etc/group | grep broadcast

# broadcast:x:7777:
```

```
# 일반 사용자 목록 보기 : 시스템 계정은 보통 UID가 0~999이고, 일반
# 사용자 목록은 UID가 1000 이상입니다.
awk -F: '$3 >= 1000 && $3 < 65534 { print $1 }' /etc/passwd
```

```
# cloudnine라는 그룹은 사용자 생성시 기본 값으로 만들어진 것입니다.
```

```
tail /etc/group | grep cloudnine
```

```
# 문제 : 일반 사용자 cloudnine를 broadcast 그룹의 멤버로 등록하시오.
```

```
# -G 옵션 : 사용자에게 추가 그룹 지정
```

```
usermod -G broadcast cloudnine
```

```
tail /etc/group | grep cloudnine
```

```
# cloudnine:x:6000:
```

```
# broadcast:x:7777:cloudnine
```

```
# 결과를 보면 현재 cloudnine라는 그룹과, broadcast이라는 그룹에 cloudnine가
# 소속이 되어 있습니다.
```

≡ User Account Management

≡ 그룹 생성과 삭제

특정 이름의 새로운 그룹을 생성하고 삭제하는 테스트를 진행해 봅니다.

문제 : pressdesk이라는 그룹을 생성해 봅니다.

```
groupadd pressdesk
```

```
tail /etc/group | grep pressdesk
```

pressdesk:x:7778:

이전에 실행했던 마지막 번호인 7777의 다음 번호, 즉 7778이 현재의 GID가 됩니다.

문제 : 시스템 관리자로 그룹 생성하기

시스템 관리용으로 1000 이하의 GID를 생성하는 경우도 있는데, 이럴 경우에는 -r 옵션을 사용합니다.

```
groupadd -r mediacrew
```

```
tail /etc/group | grep mediacrew
```

mediacrew:x:987:

≡ User Account Management

≡ 그룹 생성과 삭제

특정 이름의 새로운 그룹을 생성하고 삭제하는 테스트를 진행해 봅니다.

```
groups # 현재 접속자가 속해 있는 그룹 확인
# root

groups cloudnine # cloudnine 사용자가 속해 있는 그룹 확인
# cloudnine : cloudnine broadcast

# 일반 사용자들이 어느 그룹에 속해 있는 지 확인하기
for user in $(awk -F: '($3 >= 1000 {print $1})' /etc/passwd)
do
    echo "$user: $(groups $user)"
    echo "-----"
done
```

```
# 실행 결과
nobody : nobody : nogroup
-----
librarian : librarian : librarian adm cdrom sudo dip plugdev lxd
-----
starshine : starshine : starshine
-----
cloudnine : cloudnine : cloudnine broadcast
-----
thunderfox : thunderfox : thunderfox
-----
fireball : fireball : fireball
-----
```

Ownership&Permissions

chmod 명령어는 파일이나 디렉터리의 권한(permission)을 변경하기 위한 명령어입니다.

다음 파일들은 모든 사용자들에게 쓰기 권한이 부여 되어 있습니다.

```
ls -l /home/librarian/fiction/fantasy/
```

```
total 8
```

```
-rw-rw-r-- 1 librarian librarian 15 Apr 22 03:32 lord_of_the_rings.txt
```

```
-rw-rw-r-- 1 librarian librarian 15 Apr 22 03:32 harry_potter_summary.txt
```

lord_of_the_rings.txt 파일은 -rw-rw-r- 모드로 설정이 되어 있습니다. 즉, 수정이 가능합니다.

esc :wq 라고 실행이 오류가 없으면 잘 적용된 것입니다.

```
vi /home/librarian/fiction/fantasy/lord_of_the_rings.txt # 수정하세요.
```

수정 작업이 잘 이루어 졌는 지 확인합니다.

```
cat /home/librarian/fiction/fantasy/lord_of_the_rings.txt
```

Ownership&Permissions

chmod 명령어는 파일이나 디렉터리의 권한(permission)을 변경하기 위한 명령어입니다.

fiction/fantasy/lord_of_the_rings.txt 파일을 모든 사용자가 읽을 수 있도록 하고, 소유자만 수정할 수 있도록 권한을 설정하세요.

즉, `-rw-rw-r--`인 상태를 `-rw-r--r--` 상태로 변경해 주세요.

```
ls -al fiction/fantasy/lord_of_the_rings.txt
```

```
chmod 644 fiction/fantasy/lord_of_the_rings.txt
```

```
ls -al fiction/fantasy/lord_of_the_rings.txt
```


Ownership&Permissions

chmod 명령어는 파일이나 디렉터리의 권한(permission)을 변경하기 위한 명령어입니다.

파일 `lord\_of\_the\_rings.txt`와 `harry\_potter\_summary.txt`를 실수로 내용을 수정되지 않도록 읽기 전용으로 변경합니다.

```
chmod 444 /home/librarian/fiction/fantasy/lord_of_the_rings.txt
```

```
chmod 444 /home/librarian/fiction/fantasy/harry_potter_summary.txt
```

잘 반영이 되었는 지 다시 확인합니다.

```
ls -l /home/librarian/fiction/fantasy/
```

```
total 8
```

```
-r--r--r-- 1 librarian librarian 15 Apr 22 03:32 lord_of_the_rings.txt
```

```
-r--r--r-- 1 librarian librarian 15 Apr 22 03:32 harry_potter_summary.txt
```

lord_of_the_rings.txt 파일을 다시 수정해 보도록 하겠습니다.

```
vi /home/librarian/fiction/fantasy/lord_of_the_rings.txt
```

수정 작업을 가한 다음 :wq를 눌러 보면 다음과 같은 메시지를 볼 수 있습니다.

```
# E45: 'readonly' option is set (add ! to override)
```

```
# Press ENTER or type command to event
```

```
# :q!를 눌러 종료하도록 합니다.
```

Ownership&Permissions

chmod 명령어는 파일이나 디렉터리의 권한(permission)을 변경하기 위한 명령어입니다.

```
# `fiction_quantum_physics.txt`, `fiction_evolution_theory.txt` 파일을 찾습니다.
```

```
find /home/librarian -name "k*.txt"
```

```
/home/librarian/fiction/mystery/fiction_quantum_physics.txt
```

```
/home/librarian/fiction/mystery/fiction_evolution_theory.txt
```

```
# `fiction_quantum_physics.txt`, `fiction_evolution_theory.txt`에 실행 권한 추가
```

```
# 목적: 실행 가능한 스크립트 파일로 사용한다고 가정
```

```
# '+x' 기호는 모든 소유자들에게 실행 권한 부여
```

```
chmod +x /home/librarian/fiction/mystery/fiction_quantum_physics.txt
```

```
chmod +x /home/librarian/fiction/mystery/fiction_evolution_theory.txt
```

```
# 그룹에 대한 실행 권한 삭제
```

```
chmod g-x fiction/mystery/fiction_evolution_theory.txt
```

Ownership&Permissions

chmod 명령어는 파일이나 디렉터리의 권한(permission)을 변경하기 위한 명령어입니다.

`overview.txt` 파일들만 소유자에게만 접근 가능하게 설정합니다.

```
find /home/librarian -name "settings*.txt"
```

```
# /home/librarian/fiction/mystery/overview.txt
```

```
# /home/librarian/nonfiction/history/overview.txt
```

```
ls -l /home/librarian/fiction/mystery/overview.txt
```

```
chmod 600 /home/librarian/fiction/mystery/overview.txt
```

```
ls -l /home/librarian/nonfiction/history/overview.txt
```

```
chmod 600 /home/librarian/nonfiction/history/overview.txt
```

Ownership&Permissions

chmod 명령어는 파일이나 디렉터리의 권한(permission)을 변경하기 위한 명령어입니다.

```
# `science` 디렉토리는 읽기 전용 디렉토리로 (모든 사용자에게)
# 결과: 디렉토리 내 파일 목록은 볼 수 있지만, 파일 생성/삭제 불가
chmod 555 /home/librarian/nonfiction/science

# 결과: 디렉토리 내 파일 목록은 볼 수 있지만, 파일 생성/삭제 불가
touch /home/librarian/nonfiction/science/manage.txt
# touch: cannot touch `/home/librarian/nonfiction/science/manage.txt': Permission denied

# 누구든지 읽고, 쓰고, 실행 가능하도록 `modern` 디렉토리내의 파일 `abandon.txt`를 모든 사용자에게 완전 공개합니다.
chmod 777 /home/librarian/poetry/modern/abandon.txt

# `classic` 디렉토리는 사용자에게만 모든 접근 권한을 부여해 주세요.
chmod 700 /home/librarian/poetry/classic

# 전체 재귀적으로 읽기 전용으로 만들기 (실습용으로)
# `-R`: 하위 디렉토리까지 적용, # `a-w`: 모든 사용자에게 쓰기 권한 제거
chmod -R a-w /home/librarian/fiction
```

Ownership&Permissions

chown 명령어는 소유권을 변경하고자 하는 경우 사용하는 명령어입니다.

관리자가 drama이라는 디렉토리를 생성합니다.

```
sudo mkdir drama
```

sudo 명령어를 사용하였으므로 해당 디렉토리의 소유권은 관리자에게 있습니다.

```
ls -l
```

```
total 16
```

```
# drwxrwxr-x 4 librarian librarian 4096 Apr 22 03:30 fiction
```

```
# drwxrwxr-x 4 librarian librarian 4096 Apr 22 03:29 nonfiction
```

```
# drwxrwxr-x 4 librarian librarian 4096 Apr 22 03:37 poetry
```

```
# drwxr-xr-x 2 root root 4096 Apr 23 07:41 drama
```

접속한 일반 사용자는 drama에 대한 소유권이 없으므로 다음과 같이

파일 생성이 불가능합니다.

```
touch drama/blue.txt
```

```
# touch: cannot touch `drama/blue.txt': Permission denied
```

관리자는 drama에 대한 소유권을 현재 접속자에게 넘겨 줍니다.

```
sudo chown -R librarian:librarian drama
```

```
ls -l
```

접속한 일반 사용자는 이제 drama에 대한 소유권을 넘겨 받았으므로

다음과 같이 파일 생성이 가능합니다.

```
touch drama/blue.txt
```

```
ls drama
```

blue.txt ← 생성된 파일 목록 확인

Ownership&Permissions

chown 명령어는 소유권을 변경하고자 하는 경우 사용하는 명령어입니다.

```
# 특정 디렉토리에 대한 소유권 변경
# mystery 디렉토리 안의 모든 파일의 소유자를 cloudnine로, 그룹을
editorteam으로 변경하세요.
# 소유자와 그룹 확인
ls -al fiction/mystery/*

sudo chown cloudnine:editorteam fiction/mystery/*

# 소유자와 그룹 다시 확인
ls -al fiction/mystery/*
```

```
# 특정 파일에 대한 소유권 변경
# nonfiction/history/overview.txt의 소유자를 root로 변경하세요.
ls -al nonfiction/history/overview.txt

sudo chown root nonfiction/history/overview.txt

ls -al nonfiction/history/overview.txt
```

Ownership&Permissions

chgrp 명령어는 파일/디렉토리의 소유자/그룹 소유권을 변경하는 데 사용되는 명령어입니다.

`broadcast`라는 그룹의 정보를 확인합니다.

```
grep broadcast /etc/group
```

fiction 디렉토리 전체의 그룹 소유자를 broadcast 그룹으로 변경하세요.

힌트: chgrp -R

before : 다음 명령어 실행 결과의 GID의 정보를 기억해 두세요.

```
ls -lR fiction
```

```
sudo chgrp -R broadcast fiction
```

after : 모든 항목들의 GID가 broadcast으로 변경이 되어 있어야 합니다.

```
ls -lR fiction
```

poetry/classic 디렉토리는, 앞으로 starshine 그룹 사용자들이 콘텐츠를 추가할 예정이므로, 디렉토리의 그룹을 starshine로 설정하고, 그 그룹 구성원들이 파일을 추가할 수 있도록 쓰기 권한도 추가하세요.

```
ls -l poetry # classic의 현재 그룹 정보 확인
```

```
sudo chgrp starshine poetry/classic
```

```
ls -l poetry # classic의 GID 확인
```

classic을 자기 자신만 읽기 전용으로 설정합니다.

```
sudo chmod 400 /classic
```

```
touch aa.txt # 임의의 파일 생성
```

쓰기 권한이 없는 상태에서 쓰기 시도하기

```
cp aa.txt poetry/classic/ # Permission denied
```

```
sudo chmod 775 poetry/classic # Permission 풀어 주기
```

```
cp aa.txt poetry/classic/ # 복사가 가능해 집니다.
```

프로세스 모니터링 및 종료

프로세스를 모니터링하고 종료하는 방법에 대하여 살펴 보도록 하겠습니다.

ps, kill - 프로세스 모니터링 및 종료

`vi print_tree.sh`

다음과 같이 작성하고

`ls`

`echo 'hello~~world'`

`tree`

`:wq`를 사용하여 저장 후 종료합니다.

`ls -l print_tree.sh` # 실행 권한이 있는지 확인

`./print_tree.sh` # 실행하면 "Permission denied"이 뜸

`chmod 744 print_tree.sh`

`./print_tree.sh` # 실행하기

현재 실행 중인 print_tree.sh 프로세스의 PID를 확인하세요.

힌트: `ps -ef | grep print_tree`

`ps -ef | grep print_tree.sh`

앞의 숫자 2492가 PID, 뒤의 숫자 1106이 부모 프로세스 ID(PPID)입니다.

librarian 2492 1106 0 07:01 pts/0 00:00:00 grep --color=auto
print_tree.sh

PID가 2492인 print_tree.sh 프로세스를 종료하세요.

`kill 2492`

이건 실제 print_tree.sh가 아니라 grep 명령어입니다. 이걸 죽이려 해도 이미 kill 명령어를 입력하는 시점에는 grep은 이미 종료된 상태이므로 "No such process"가 나옵니다.

강제로 종료하고자 하는 경우에는 `'kill -9'` 명령어를 사용하면 됩니다

`kill -9 2492`

≡ User Account Management

≡ 그룹 생성과 삭제

특정 이름의 새로운 그룹을 생성하고 삭제하는 테스트를 진행해 봅니다.

```
# 일반 사용자들이 어느 그룹에 속해 있는 지 확인하기
for user in $(awk -F: '($3 >= 1000 {print $1})' /etc/passwd)
do
    echo "$user: $(groups $user)"
done

# 문제 : 지금까지 생성한 그룹을 모두 삭제하시오.
groupdel editorteam

groupdel broadcast

groupdel pressdesk

tail /etc/group
```

리눅스(Linux)

파일 압축/해제



파일 압축/해제

wget 명령어를 사용하여 tar 파일을 압축/압축 해제하는 방법에 대하여 살펴 봅니다.

```
# Oracle 공식 Java JDK 17의 Linux용 아카이브 다운로드
# https://download.oracle.com/java/17/archive/jdk-17.0.12\_linux-aarch64\_bin.tar.gz

# wget 명령어를 사용한 다운로드 및 압축 해제 실습을 진행합니다.
# 우선 실습을 위한 작업 디렉토리를 생성한 다음 해당 디렉토리로 이동합니다.
mkdir -p ~/compress/wgetexam
cd ~/compress/wgetexam

ls # 파일 없음

# tar.gz 파일 다운로드
wget https://download.oracle.com/java/17/archive/jdk-17.0.12\_linux-aarch64\_bin.tar.gz

ls # 파일 보임
jdk-17.0.12\_linux-aarch64\_bin.tar.gz

tar -xzf jdk-17.0.12\_linux-aarch64\_bin.tar.gz # 압축 해제
```

파일 압축/해제

curl 명령어를 사용하여 tar 파일을 압축/압축 해제하는 방법에 대하여 살펴 봅니다.

```
# 홈 디렉토리로 이동하기
```

```
cd
```

```
# curl 명령어를 사용한 다운로드 및 압축 해제
```

```
# 작업 디렉토리 생성 및 이동
```

```
mkdir -p ~/compress/curlexam
```

```
cd ~/compress/curlexam
```

```
# jdk.tar.gz는 다운로드시 지정할 파일 이름
```

```
curl -L -o jdk.tar.gz https://download.oracle.com/java/17/archive/jdk-17.0.12_linux-aarch64_bin.tar.gz
```

```
# 압축 해제
```

```
tar -xzf jdk.tar.gz
```

```
tree -L 2
```

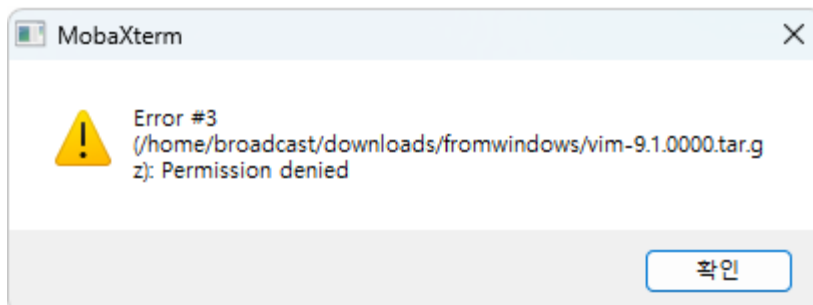
파일 압축/해제

tar 형식의 압축 파일을 압축하거나 압축 해제하는 방법에 대하여 살펴 봅니다.

```
# window에서 다운로드 받은 파일 압축하기  
sudo mkdir -p ~/compress/fromwindows
```

```
cd ~/compress/fromwindows
```

파일을 복사하기 위하여 Drag And Drop 기능을 이용하면 됩니다. 하지만 복사가 되지 않습니다. why?



```
cd
```

```
ls -al /home/librarian/compress
```

```
# fromwindows 권한 및 소유권 확인
```

```
sudo chown -R librarian:librarian /home/librarian/compress/fromwindows/
```

```
cd compress/fromwindows/
```

```
# 윈도우에서 파일에 대하여 Drag And Drop 기능을 사용하여 복사합니다.
```

```
# 파일 목록을 확인합니다.
```

```
ls
```

```
tar -xzf jdk-17.0.12_linux-aarch64_bin.tar.gz # 압축 해제
```

파일 압축/해제

특정 파일이나 디렉터리에 대하여 압축을 하거나 해제하는 방법에 대하여 살펴 봅니다.

```
# zip 명령어 사용하기  
# fiction\mystery와 nonfiction\science 디렉토리를  
# poetry\classic 디렉토리로 복사합니다.  
# poetry\classic 디렉토리내의 모든 파일을 압축 해제합니다.
```

```
cd # 사용자 홈 디렉토리로 이동  
sudo apt -y install zip # zip 설치  
sudo chown -R librarian fiction/mystery
```

```
# 파일 압축하고 이동하기  
cd fiction/mystery/  
zip -r mystery.zip .  
mv mystery.zip /home/librarian/poetry/classic/  
  
cd nonfiction/science/  
zip -r science.zip .  
mv science.zip /home/librarian/poetry/classic/
```

디렉토리

fiction\mystery

디렉토리

nonfiction\science

압축 후 이동

디렉토리

poetry\classic

≡ 파일 압축/해제

특정 파일이나 디렉터리에 대하여 압축을 하거나 해제하는 방법에 대하여 살펴 봅니다.

```
# 해당 디렉토리로 이동하기
cd /home/librarian/poetry/classic/

# 파일 목록을 확인합니다.
ls
# science.zip  mystery.zip

unzip science.zip
unzip mystery.zip

ls
# quantum_physics.txt      fiction_quantum_physics.txt  mystery.zip  evolution_theory.txt
# science.zip  fiction_evolution_theory.txt  overview.txt      guide.txt
```

파일/디렉토리 삭제

특정 파일이나 디렉터리에 대하여 **삭제하는 방법**에 대하여 살펴 봅니다.

```
# wgetexam 디렉토리 내의 파일 또는 디렉토리 삭제 테스트입니다.
```

```
cd compress/wgetexam/
```

```
ls
```

```
jdk-17.0.12  jdk-17.0.12_linux-aarch64_bin.tar.gz
```

```
rm jdk-17.0.12_linux-aarch64_bin.tar.gz
```

```
ls
```

```
jdk-17.0.12
```

```
rm jdk-17.0.12/
```

```
rm: cannot remove 'jdk-17.0.12/': Is a directory
```

```
# 디렉토리 및 하위 목록을 삭제하려면 -rf 옵션을 사용하면 됩니다.
```

```
rm -rf jdk-17.0.12/
```


파일/디렉토리 삭제

특정 파일이나 디렉터리에 대하여 **삭제하는 방법**에 대하여 살펴 봅니다.

```
cd .. # 상위 디렉토리로 이동
```

```
ls
```

```
curlexam fromwindows wgetexam
```

```
# wgetexam 디렉토리와 하위 목록을 삭제합니다.
```

```
rm -rf wgetexam/
```

```
cd
```

```
rm -rf compress/
```

```
ls # compress 디렉토리는 보이지 않아야 합니다.
```

```
# drama 디렉토리와 하위 목록을 삭제합니다.
```

```
rm -rf drama/
```

리눅스(Linux)

alias



`alias(알리아스)

긴 문장의 명령어나 복잡한 명령어 시퀀스를 단축된 별명으로 정의할 수 있게 해줍니다.

```
cat oracle.txt
```

```
find /home/librarian -name "overview.txt"
```

```
alias catoracle='cat oracle.txt'
```

```
alias finddata='find /home/librarian -name "overview.txt"
```

```
catoracle
```

```
finddata
```

```
# 설정된 알리아스 목록 확인
```

```
alias
```

```
# MobaXTerm 접속 종료 후 다시 접속하기
```

```
alias # 다시 확인하면 항목이 보이나요?
```

```
alias catoracle='cat oracle.txt'
```

```
alias finddata='find /home/librarian -name "overview.txt"
```

```
# unalias 명령은 해제할 때 사용합니다.
```

```
unalias catoracle
```

```
catoracle
```

```
Command 'catoracle' not found, did you mean:
```

```
command 'head' from deb coreutils (9.4-3ubuntu6.1)
```

```
unalias finddata
```

```
finddata
```

```
Command 'finddata' not found, did you mean:
```

```
command 'tail' from deb coreutils (9.4-3ubuntu6.1)
```

`alias(알리아스)

별도의 .sh 파일로 관리하는 방법에 대해 살펴 보도록 합니다.

cd

.sh 파일을 생성합니다.

vi alias_exam.sh

alias catoracle='cat oracle.txt'

alias finddata='find /home/librarian -name
"overview.txt"'

alias treeview='tree -L 1 /home/librarian'
:wq

source 명령어를 사용하여 실행시킵니다.

source alias_exam.sh

catoracle # 오류 없이 잘 동작함

finddata # 오류 없이 잘 동작함

fiction 디렉토리를 확인하는 명령어

fictionview라는 이름으로 'ls ~/fiction' 추가하기

vi alias_exam.sh

alias fictionview='ls ~/fiction' 내용 추가
:wq

source alias_exam.sh

fictionview

MobaXTerm 접속 종료 후 다시 접속하기

fictionview # 다시 확인하면 항목이 보이나요?

.bashrc는 리눅스에서 Bash 셸이 시작될 때 자동으로 실행되는 개인 설정 파일입니다.

vi .bashrc

alias_exam.sh 파일에 작성했던 내용을 모두 작성합니다.

MobaXTerm 접속 종료 후 다시 접속하기

fictionview # 문제 없이 실행이 되어야 합니다.

리눅스(Linux)

자바 테스트



JDK/JDBC 드라이버

자바를 위한 JDK와 JDBC 드라이버를 다운로드 합니다.

```
sudo apt update # 패키지를 최신으로 업데이트
```

```
# 자바 설치 : ubuntu나 Debian 계열 리눅스에서 자바 개발 키트 (JDK)를 설치하는 명령어입니다.
```

```
# sudo apt install default-jdk
```

```
sudo apt install -y openjdk-17-jdk # 스프링 부트 버전에 맞도록 설치하면 됩니다.
```

```
# 설치 이후 자바 버전을 확인하세요:
```

```
java -version
```

```
# openjdk version "17.0.14" 2025-01-21
```

```
# OpenJDK Runtime Environment (build 17.0.14+7-ubuntu-120.04)
```

```
# OpenJDK 64-Bit Server VM (build 17.0.14+7-ubuntu-120.04, mixed mode, sharing)
```

```
# MySQL JDBC 드라이버(Jar 파일)를 인터넷에서 다운로드하는 명령어입니다.
```

```
wget https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.4.0/mysql-connector-j-8.4.0.jar
```

Java 코드 작성

JDBC 드라이버 테스트를 위한 자바 코드를 다음과 같이 작성합니다.

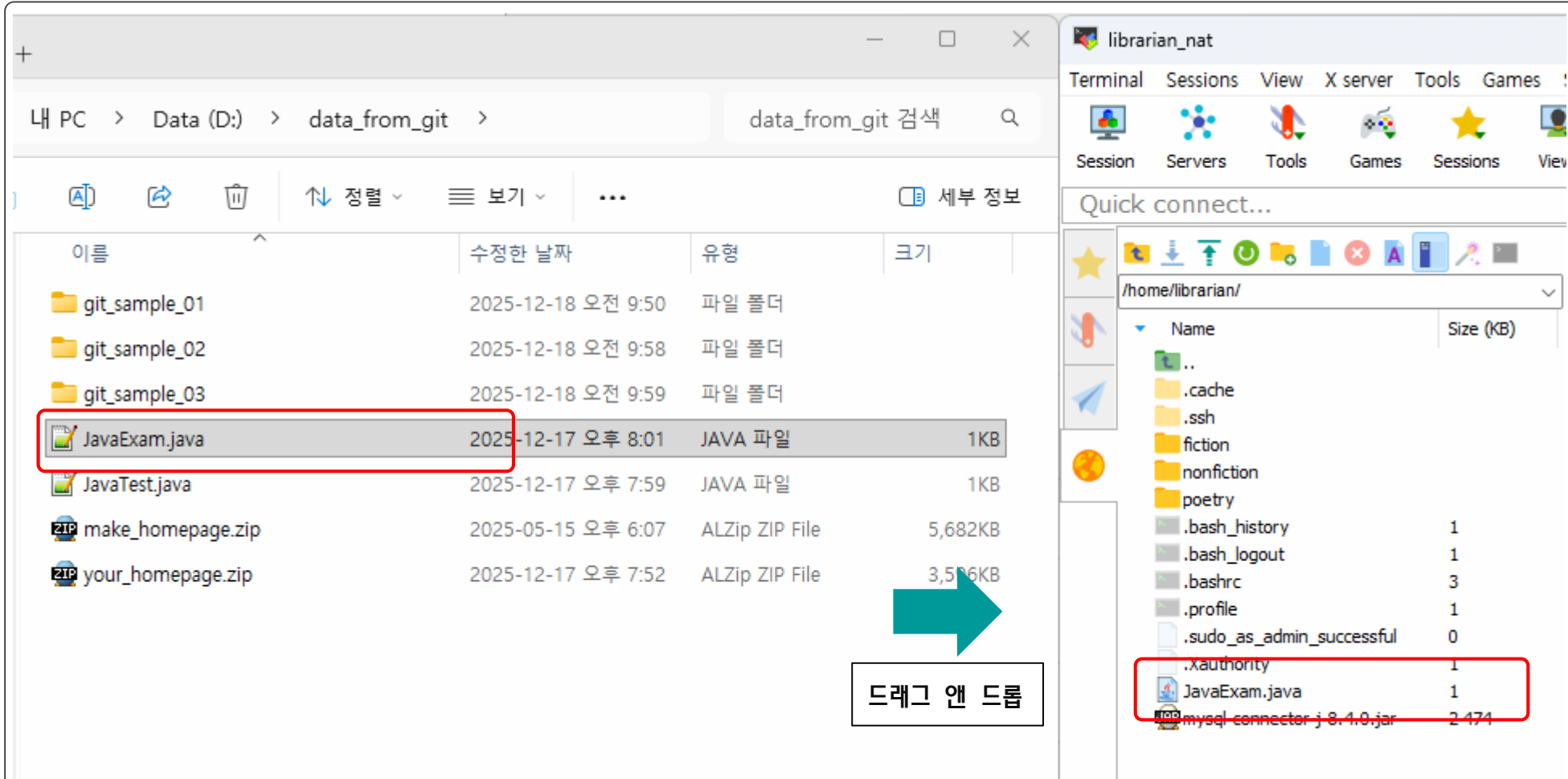
JavaExam.java

소스 코드

```
// import java.sql.Connection;  
// import com.mysql.cj.jdbc.Driver;  
  
public class JavaExam {  
    public static void main(String[] args) {  
        int su01 = 3 ;  
        int su02 = 5 ;  
        System.out.println(su01 + " 더하기 " + su02 + "는 " + (su01 + su02) + "입니다.");  
    }  
}
```

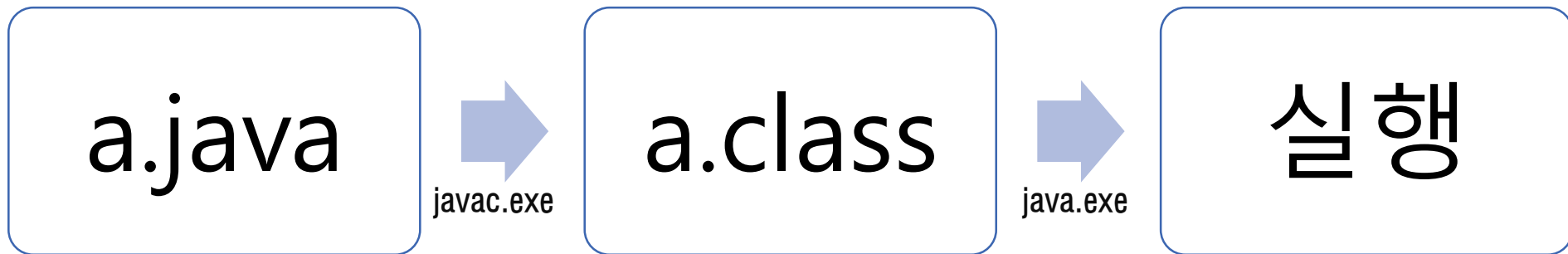
Java 코드 작성

작성한 자바 코드를 MobaXTerm에 DragAndDrop 기능을 사용하여 복사합니다.



컴파일과 실행하기

자바 코드에 대하여 다음과 같이 컴파일을 진행한 다음 실행하도록 합니다.



```
ls J* # 대문자 J로 시작하는 항목 조회
```

```
JavaExam.java
```

```
# 컴파일과 실행하기 : -cp 옵션은 클래스 경로(classpath)를 지정하는 옵션 (외부 라이브러리 포함 시 사용)
```

```
# 이 명령을 실행하면 JavaExam.class Binary 파일이 생성됩니다.
```

```
javac -cp ./mysql-connector-j-8.4.0.jar JavaExam.java
```

```
# 현재 디렉토리와 MySQL 드라이버 JAR 파일을 클래스 경로에 포함시켜서 해당 클래스 파일을 실행해 주세요.
```

```
java -cp ./mysql-connector-j-8.4.0.jar JavaExam
```

```
# 실행 결과는 다음과 같습니다.
```

```
# 3 더하기 5는 8입니다.
```

리눅스(Linux)

웹 서버 구동



≡ Apache WebServer 구동

Ubuntu에서 Apache WebServer 관련 구동 테스트를 해도보록 하겠습니다.

시스템에 설치된 모든 패키지를 최신 버전으로 업그레이드합니다.

```
sudo apt update
```

```
sudo apt install -y apache2 # 조금 시간이 걸립니다.
```

systemctl은 System Control의 줄인 말로 서비스 실행/중지 등의 작업을 수행하도록 해주는 명령어입니다.

```
sudo systemctl enable apache2 # enable은 시스템 부팅 시 자동으로 시작되도록 설정하는 명령어입니다.
```

```
sudo systemctl start apache2
```

```
sudo systemctl status apache2
```

다음과 같은 문구가 보이는 지 확인합니다.

Active: **active (running)** since Thu 2025-04-03 12:05:52 UTC; 1min 12s ago

... 이하 중략

Ctrl + C를 눌러서 종료 합니다.

Apache WebServer 구동

Ubuntu인 경우 `ufw`(Uncomplicated Firewall)가 기본적으로 차단이 되어 있을 수 있습니다.

소스 코드

인스턴스 내부 방화벽으로 인하여 테스트가 잘 진행되지 않으면 차단을 해제해 주어야 합니다.

다음 명령어를 순차적으로 실행하면 됩니다.

```
sudo ufw status
```

```
Status: inactive
```

만약 방화벽이 활성화되어 있지 않다면 다음 명령어를 실행합니다.

```
sudo ufw enable
```

```
Command may disrupt existing ssh connections. Proceed  
with operation (y/n)? y
```

```
Firewall is active and enabled on system startup
```

```
sudo ufw status
```

```
# Status: active 이면서 80, 443 포트가 없으면 차단된 상태입니다.
```

```
Status: active
```

다음과 같이 실습하도록 합니다.

```
sudo ufw allow 80/tcp
```

```
Rules updated
```

```
Rules updated (v6)
```

```
sudo ufw allow 443/tcp
```

```
Rules updated
```

```
Rules updated (v6)
```

```
sudo ufw allow 22/tcp
```

```
# 다른 방법으로 sudo ufw allow ssh
```

Apache WebServer 구동

Ubuntu의 경우 `ufw`(Uncomplicated Firewall)가 기본적으로 차단이 되어 있을 수 있습니다.

```
sudo ufw enable
```

```
sudo ufw status
```

Status: **active**

Status: active 이면서 80, 443 포트 정보가 아래에 존재하면 잘 설정된 것입니다.

To	Action	From
--	-----	----
80/tcp	ALLOW	Anywhere
443/tcp	ALLOW	Anywhere
80/tcp (v6)	ALLOW	Anywhere (v6)
443/tcp (v6)	ALLOW	Anywhere (v6)

pc를 재부팅합니다.

```
sudo reboot # VirtualBox에서 부팅이 되는 지 확인하도록 합니다.
```

재부팅 이후 MobaXTerm으로 접속이 잘 되지 않으면 다음 명령어를 추가적으로 작성해 봅니다.

```
sudo ufw allow ssh # redux mobaxterm
```

```
sudo ufw allow 80/tcp # redux http
```

```
sudo ufw allow 443/tcp # redux https
```

```
sudo ufw reload
```

```
sudo ufw status # ssh 목록 2개 보임
```

MobaXTerm으로 다시 접속 후 다음 2개의 항목이 active인지 확인 요망

```
sudo systemctl status apache2
```

```
sudo systemctl status ssh
```

VirtualBox에서 80번 포트 포워딩 on

Apache WebServer 구동

홈 페이지 접속을 위하여 윈도우의 브라우저에서 다음과 같이 입력해 봅니다.



Apache2 Default Page

Ubuntu

It works!

This is the default welcome page used to test the correct operation of the Apache2 installation on Ubuntu systems. It is based on the equivalent page on Debian, from which the Ubuntu Apache packaging is derived. If you can read this page, it means that the Apache HTTP server installed at this site is working properly. You should **replace this file** (located at `/var/www/html/index.html`) before continuing to operate your HTTP server.

If you are a normal user of this web site and don't know what this page is about, this probably means that the site is currently unavailable due to maintenance. If the problem persists, please contact the site's administrator.

Configuration Overview

Ubuntu's Apache2 default configuration is different from the upstream default configuration, and split into several files optimized for interaction with Ubuntu tools. The configuration system is **fully documented in `/usr/share/doc/apache2/README.Debian.gz`**. Refer to this for the full documentation. Documentation for the web server itself can be found by accessing the **manual** if the `apache2-doc` package was installed on this server.

The configuration layout for an Apache2 web server installation on Ubuntu systems is as follows:

입력 url

윈도우 컴퓨터에서 웹 브라우저에 다음과 같이 입력해 봅니다.

항목	설명
	http://localhost:8200
Nat	여기서 8200은 VirtualBox에서 설정한 포트
Bridge	http://aa.bb.cc.dd

Apache WebServer 구동

Ubuntu에서 Apache WebServer 관련 구동 테스트를 해도보록 하겠습니다.



Apache WebServer 구동

내가 만든 홈 페이지를 실행시키기 위하여 your_homepage.zip 파일로 다음과 같이 셋팅합니다.

your_homepage.zip

```
cd
mkdir yourhomepage
cd yourhomepage
unzip your_homepage.zip # 윈도우에서 리눅스로 파일 복사
mv index04.html index.html

ls -l /var/www/html/ # 아파치 홈 디렉토리 확인.
sudo cp /var/www/html/index.html /var/www/html/index.bak

# 아파치를 설치한 경우 기본 웹 문서 디렉터리 (DocumentRoot)는 '/var/www/html/'입니다.
sudo cp -r ./* /var/www/html/
[sudo] password redux librarian: 비밀 번호입력
ls -al /var/www/html/
# total 3460
# drwxr-xr-x 2 root root    4096 May 10 12:06 .
# -rw-r--r-- 1 root root    733 May 10 12:06 index.html
# ... 이하 중략
```

your_homepage.zip

파일 내역
index04.html
image17_02.png

Apache WebServer 구동

아파치를 중지 후 재시작 테스트를 수행해 봅니다.

your_homepage.zip

```
sudo systemctl stop apache2  
# http://localhost에서 접속이 안됨  
  
sudo systemctl start apache2  
# http://localhost에서 접속이 잘 됨
```



사이트에 연결할 수 없음

연결이 재설정되었습니다.

다음 방법을 시도해 보세요.

- 연결 확인
- 프록시 및 방화벽 확인
- Windows 네트워크 진단 프로그램 실행

ERR_CONNECTION_RESET

새로고침

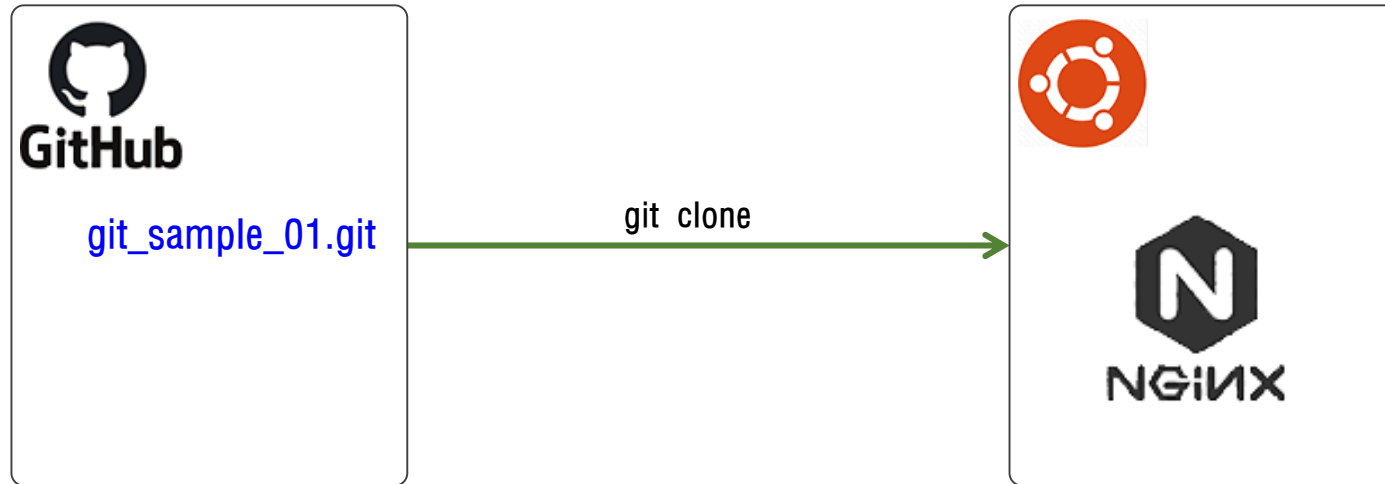
귤의 정원 바령

제주도 ~ 2022년도



☰ Nginx 구동

Ubuntu에서 엔진 X 관련 구동 테스트를 해도보록 하겠습니다.



☰ Nginx 구동

Ubuntu에서 엔진 X 관련 구동 테스트를 해도보록 하겠습니다.

시스템에 설치된 모든 패키지를 최신 버전으로 업그레이드합니다.

```
sudo apt update
```

```
sudo apt install -y nginx
```

```
sudo systemctl start nginx
```

```
sudo systemctl status nginx
```

Nginx 구동

Ubuntu에서 엔진 X 관련 구동 테스트를 해도보록 하겠습니다.

`exit` # 관리자로 로그인 했을 시 사용자 모드로 변경하는 명령어

`git clone https://github.com/seoljinuk/git_sample_01.git`

`ls` # 목록 확인

`cd git_sample_01/Public-2C/`

`ls`

`sudo cp -r ./* /var/www/html/`

입력 url

윈도우 컴퓨터에서 웹 브라우저에 다음과 같이 입력해 봅니다.

포트 번호 8200은 VirtualBox에서 설정한 값입니다.

<http://localhost:8200>

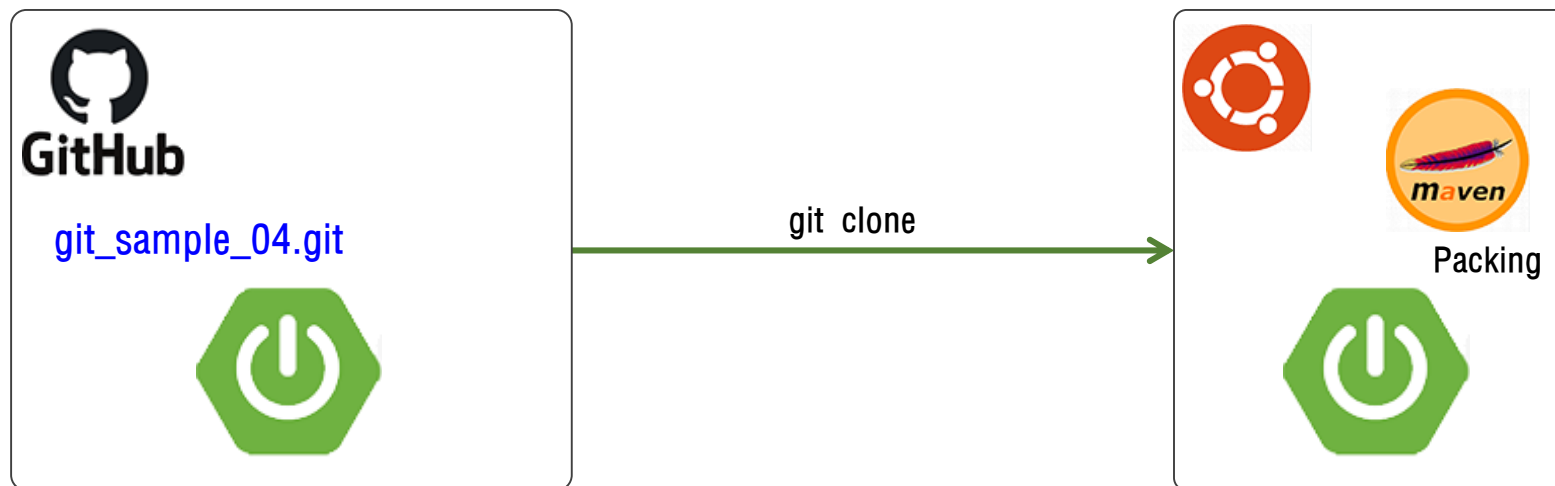
웹 서버가 정상적으로 동작합니다!

인스턴스에서 웹 서버 테스트 페이지입니다.



스프링 부트와의 연동

스프링 부트에서 작성한 파일을 패키징하여 구현해 보도록 하겠습니다.



요구 사항

항목	설명
호스트 아이피	8200
스프링 포트 번호	8500

☰ 스프링 부트와의 연동

스프링 부트에서 작성한 파일을 패키징하여 구현해 보도록 하겠습니다.

```
# 이전에 nginx를 실습하였다면 다음 구문들을 실행시켜 주세요.  
sudo systemctl stop nginx # 구동중인 nginx 종료 하기  
sudo systemctl disable nginx # 재부팅시 구동이 안되도록 지정하기  
  
git clone https://github.com/seoljinuk/git_sample_04.git  
cd git_sample_04/  
grep port src/main/resources/application.properties # 포트 번호 확인하기  
# server.port=9000  
  
vi src/main/resources/application.properties  
:s/9000/8500/g  
:wq 으로 저장 후 종료하기
```

☰ 스프링 부트와의 연동

스프링 부트에서 작성한 파일을 패키징하여 구현해 보도록 하겠습니다.

포트 포워딩(포트 리디렉션) : 8500를 80포트로 ...

```
sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8500
```

설정이 잘 되었는 가 확인하기 위하여 다음 문장을 실행하면 아래와 같은 항목이 보여야 합니다.

```
sudo iptables -t nat -L -n
```

```
# REDIRECT 6 -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 redir ports 8500
```

주의) 8500번 포트의 우분투 방화벽 열어 주기

```
sudo ufw allow 8500/tcp
```

Rule added

Rule added (v6)

≡ 스프링 부트와의 연동

스프링 부트에서 작성한 파일을 패키징하여 구현해 보도록 하겠습니다.

```
# 메이븐 설치하기
sudo apt install -y maven
mvn -v

# 스프링 부트 소스 전체 빌드(컴파일 + 테스트 + 패키징)
# 주의) 반드시 pom.xml 파일이 있는 곳에서 실행
mvn clean package -DskipTests

# jar 파일이 생성되는 디렉토리 위치
cd target/

# 스프링 부트 애플리케이션 실행
java -jar shopping-0.0.1-SNAPSHOT.jar
```


스프링 부트와의 연동

브라우저에서 다음과 같이 결과를 확인합니다.

상품 목록



귤의 정원
4,500원



아메리카노
5,000원



상세 페이지



귤의 정원

가격: 4,500원

설명: 제주도 귤의 정원 체럼

목록으로